```
!pip install tensorflow tensorflow-datasets faiss-cpu numpy matplotlib seaborn
```

```
import os
import time
import json
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow_datasets as tfds

sns.set_theme(style="whitegrid")
plt.rcParams.update({'font.size': 12, 'figure.figsize': (10, 6)})
```

```
ds_base = tfds.load("sift1m", split="database", as_supervised=False)
ds_test = tfds.load("sift1m", split="test", as_supervised=False)
```

```
WARNING:absl:Variant folder /root/tensorflow_datasets/sift1m/1.0.0 has no dataset_info.json
Downloading and preparing dataset Unknown size (download: Unknown size, generated: Unknown size, total: Unknown s
Dl Completed...: 100%  1/1 [00:12<00:00, 12.59s/ url]
Dl Size...: 100%  500/500 [00:12<00:00, 36.98 MiB/s]
Extraction completed...:  0/0 [00:12<?, ? file/s]
Dataset sift1m downloaded and prepared to /root/tensorflow_datasets/sift1m/1.0.0. Subsequent calls will reuse thi
```

```
for example in ds_base.take(1):
    print(example.keys())
```

```
dict_keys(['embedding', 'index', 'neighbors'])
```

```
base_vectors = np.array([x['embedding'] for x in tfds.as_numpy(ds_base)], dtype='float32')
query_vectors = np.array([x['embedding'] for x in tfds.as_numpy(ds_test)], dtype='float32')

print(f"Database: {base_vectors.shape}")
print(f"Query: {query_vectors.shape}")
```

```
Database: (1000000, 128)
Query: (10000, 128)
```

3. Для каждого вектора-запроса в исходном датасете при помощи kNN составить списки точных 100 соседей - их считаем ground truth

Предустановленный Ground Truth

```
gt_precomputed = []
for x in tfds.as_numpy(ds_test):
    gt_precomputed.append(x['neighbors']['index'])

gt_precomputed = np.array(gt_precomputed, dtype='int32')

if gt_precomputed.shape[1] > 100:
    gt_precomputed = gt_precomputed[:, :100]

print(f"Ground Truth: {gt_precomputed.shape}")
```

```
Ground Truth: (10000, 100)
```

4. Выбрать любую библиотеку в которой уже есть готовые реализации LSH/HNSW/IVF+PQ

```
import faiss
```

Ground Truth вручную

```
index_flat = faiss.IndexFlatL2(128)
index_flat.add(base_vectors)

start_time = time.time()
distance, gt_manual = index_flat.search(query_vectors, 100)

sample_idx = 0
```

```
matches = np.intersect1d(gt_manual[sample_idx], gt_precomputed[sample_idx]).size
print(f"Совпадение для запроса №0: {matches}/100")
```

```
Совпадение для запроса №0: 100/100
```

```
# gt_final = gt_manual
gt_final = gt_precomputed
```

```
Напишите программный код или сгенерируйте его с помощью искусственного интеллекта.
```

Функция оценки Recall

```
def evaluate_index(index, queries, ground_truth, k=100):
    start_time = time.time()
    distances, indices = index.search(queries, k)
    end_time = time.time()

    total_queries = ground_truth.shape[0] # 10 000
    correct_hits = 0

    for i in range(total_queries):
        intersect_count = np.intersect1d(indices[i], ground_truth[i]).size
        correct_hits += intersect_count

    recall = correct_hits / (total_queries * k)

    total_time = end_time - start_time
    qps = total_queries / total_time

    return recall, qps
```

LSH

- nbits - количество бит в хэше

```
lsh_results = []
dimension = 128

for nbits in [512, 1024, 2048]:
    print(f"LSH с nbits={nbits}")

    index_lsh = faiss.IndexLSH(dimension, nbits)

    start_build = time.time()
    index_lsh.add(base_vectors)
    build_duration = time.time() - start_build

    recall, qps = evaluate_index(index_lsh, query_vectors, gt_final)

    lsh_results.append({
        'nbits': nbits,
        'recall': recall,
        'qps': qps,
        'build_time': build_duration
    })

    print(f"  Результат: Recall = {recall:.4f}, QPS = {qps:.1f}, Построение = {build_duration:.2f}с")
```

```
LSH с nbits=512
  Результат: Recall = 0.4561, QPS = 196.4, Построение = 4.56с
LSH с nbits=1024
  Результат: Recall = 0.5857, QPS = 56.1, Построение = 12.35с
LSH с nbits=2048
  Результат: Recall = 0.6949, QPS = 27.8, Построение = 16.80с
```

HNSW

- m - количество связей для каждого узла
- ef_value - размер списка кандидатов на 100 ближайших соседей

```
hnsw_results = []
dimension = 128
```

```
for m_value in [16, 32]:
    print(f"HNSW c M={m_value}")

    index_hnsw = faiss.IndexHNSWFlat(dimension, m_value)

    start_build = time.time()
    index_hnsw.add(base_vectors)
    build_duration = time.time() - start_build

    for ef_value in [100, 200, 500]:
        index_hnsw.hnsw.efSearch = ef_value

        recall, qps = evaluate_index(index_hnsw, query_vectors, gt_final)

        hnsw_results.append({
            'M': m_value,
            'efSearch': ef_value,
            'recall': recall,
            'qps': qps,
            'build_time': build_duration
        })

        print(f"Результат: efSearch={ef_value:3} | Recall: {recall:.4f} | QPS: {qps:.1f}, Построение = {build_du
```

```
HNSW c M=16
Результат: efSearch=100 | Recall: 0.8641 | QPS: 1996.8, Построение = 179.60c
Результат: efSearch=200 | Recall: 0.9407 | QPS: 1519.9, Построение = 179.60c
Результат: efSearch=500 | Recall: 0.9845 | QPS: 516.3, Построение = 179.60c
HNSW c M=32
Результат: efSearch=100 | Recall: 0.9548 | QPS: 1214.9, Построение = 367.72c
Результат: efSearch=200 | Recall: 0.9866 | QPS: 700.7, Построение = 367.72c
Результат: efSearch=500 | Recall: 0.9981 | QPS: 289.5, Построение = 367.72c
```

IVF+PQ

- nlist_options: на сколько кластеров делим базу
- m_options: на сколько частей режем вектор для сжатия
- nprobe_options: сколько ближайших кластеров проверять при поиске

```
ivfpq_results = []
dimension = 128

m_options = [32, 64, 128]
nlist_options = [1024, 2048, 4096]
nprobe_options = [50, 100, 500]

for m in m_options:
    for nlist in nlist_options:
        print(f"Clusters={nlist}, m={m}")

        quantizer = faiss.IndexFlatL2(dimension)
        index_ivf = faiss.IndexIVFPQ(quantizer, dimension, nlist, m, 8)

        start_train = time.time()
        index_ivf.train(base_vectors)
        train_time = time.time() - start_train

        start_add = time.time()
        index_ivf.add(base_vectors)
        add_time = time.time() - start_add

        build_time_total = train_time + add_time


        print(f"Построено за {build_time_total:.2f}c (Train: {train_time:.1f}c)")

        for nprobe in nprobe_options:
            index_ivf.nprobe = nprobe

            recall, qps = evaluate_index(index_ivf, query_vectors, gt_final)

            ivfpq_results.append({
                'algorithm': 'IVF+PQ',
                'nlist': nlist,
                'm': m,
                'nprobe': nprobe,
                'recall': recall,
```

```
                'qps': qps,
                'build_time': build_time_total
            })

            print(f"Реезультаты: nprobe={nprobe:3} | Recall: {recall:.4f} | QPS: {qps:.1f}")
```

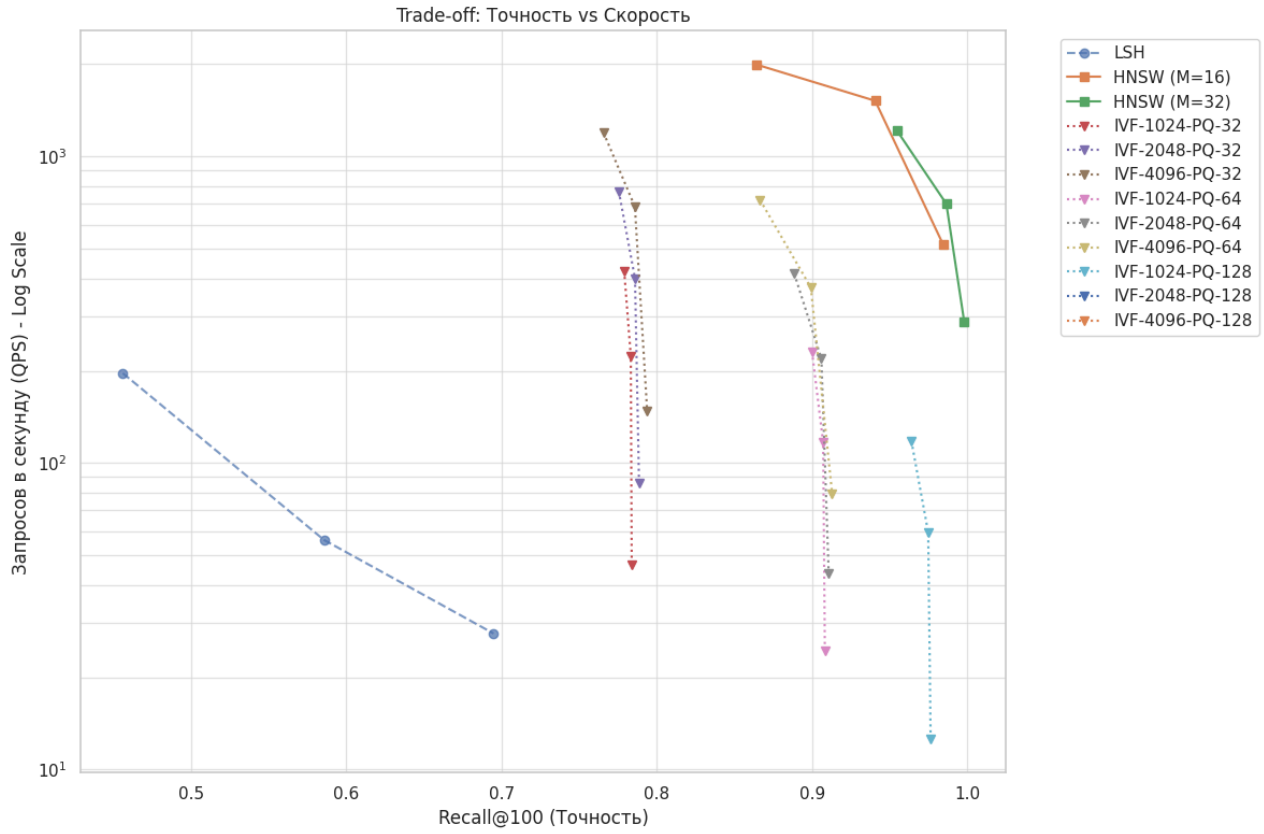
```
Clusters=1024, m=32
Построено за 34.24c (Train: 21.5c)
Реезультаты: nprobe= 50 | Recall: 0.7791 | QPS: 423.5
Реезультаты: nprobe=100 | Recall: 0.7833 | QPS: 223.7
Реезультаты: nprobe=500 | Recall: 0.7839 | QPS: 46.5
Clusters=2048, m=32
Построено за 87.24c (Train: 69.1c)
Реезультаты: nprobe= 50 | Recall: 0.7756 | QPS: 770.5
Реезультаты: nprobe=100 | Recall: 0.7860 | QPS: 399.9
Реезультаты: nprobe=500 | Recall: 0.7886 | QPS: 86.1
Clusters=4096, m=32
Построено за 265.22c (Train: 236.0c)
Реезультаты: nprobe= 50 | Recall: 0.7658 | QPS: 1195.8
Реезультаты: nprobe=100 | Recall: 0.7860 | QPS: 686.2
Реезультаты: nprobe=500 | Recall: 0.7939 | QPS: 147.7
Clusters=1024, m=64
Построено за 40.75c (Train: 27.3c)
Реезультаты: nprobe= 50 | Recall: 0.9001 | QPS: 230.9
Реезультаты: nprobe=100 | Recall: 0.9072 | QPS: 116.7
Реезультаты: nprobe=500 | Recall: 0.9083 | QPS: 24.4
Clusters=2048, m=64
Построено за 93.12c (Train: 74.5c)
Реезультаты: nprobe= 50 | Recall: 0.8886 | QPS: 414.6
Реезультаты: nprobe=100 | Recall: 0.9061 | QPS: 219.1
Реезультаты: nprobe=500 | Recall: 0.9108 | QPS: 43.7
Clusters=4096, m=64
Построено за 273.77c (Train: 243.6c)
Реезультаты: nprobe= 50 | Recall: 0.8666 | QPS: 721.8
Реезультаты: nprobe=100 | Recall: 0.8993 | QPS: 373.6
Реезультаты: nprobe=500 | Recall: 0.9130 | QPS: 79.6
Clusters=1024, m=128
Построено за 535.87c (Train: 286.1c)
Реезультаты: nprobe= 50 | Recall: 0.9641 | QPS: 118.0
Реезультаты: nprobe=100 | Recall: 0.9749 | QPS: 59.4
Реезультаты: nprobe=500 | Recall: 0.9765 | QPS: 12.6
Clusters=2048, m=128
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_1546/3062467550.py in <cell line: 0>()
     14
     15         start_train = time.time()
---> 16         index_ivf.train(base_vectors)
     17         train_time = time.time() - start_train
     18

                                 1 frames
/usr/local/lib/python3.12/dist-packages/faiss/swigfaiss.py in train(self, n, x)
   7016     def train(self, n, x):
   7017         r"""Trains the quantizer and calls train_encoder to train sub-quantizers"""
-> 7018         return _swigfaiss.IndexIVF_train(self, n, x)
   7019
   7020     def add(self, n, x):

KeyboardInterrupt:
```

Напишите программный код или сгенерируйте его с помощью искусственного интеллекта.

```
import pandas as pd

df_lsh = pd.DataFrame(lsh_results)
df_hnsw = pd.DataFrame(hnsw_results)
df_ivf = pd.DataFrame(ivfpq_results)

df_lsh['label'] = 'LSH'
df_hnsw['label'] = 'HNSW (M=' + df_hnsw['M'].astype(str) + ')'
df_ivf['label'] = 'IVF' + df_ivf['nlist'].astype(str) + '+PQ(m=' + df_ivf['m'].astype(str) + ')'
```

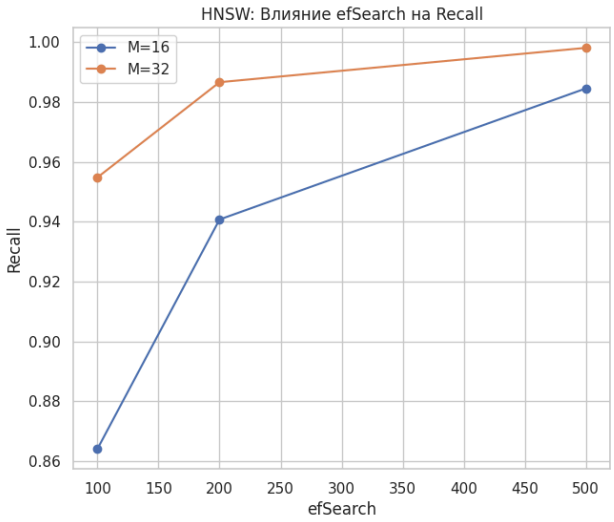
```
plt.figure(figsize=(12, 8))

plt.plot(df_lsh['recall'], df_lsh['qps'], 'o--', label='LSH', alpha=0.7)

for m in df_hnsw['M'].unique():
    subset = df_hnsw[df_hnsw['M'] == m]
    plt.plot(subset['recall'], subset['qps'], 's-', label=f'HNSW (M={m})')
```

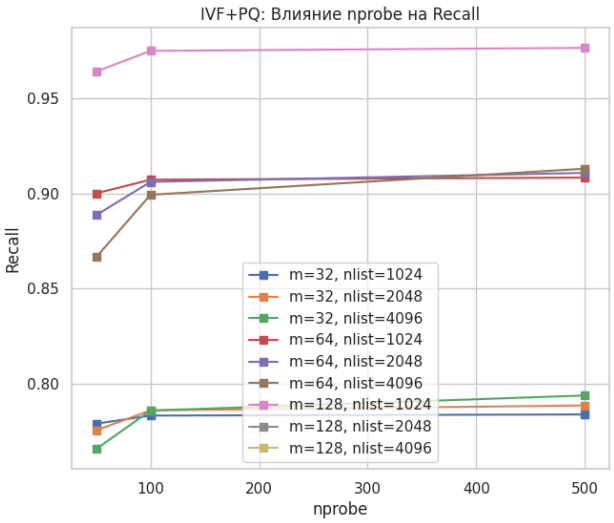
```
for m_pq in df_ivf['m'].unique():
    for nlist in df_ivf['nlist'].unique():
        subset = df_ivf[(df_ivf['m'] == m_pq) & (df_ivf['nlist'] == nlist)]
        plt.plot(subset['recall'], subset['qps'], 'v:', label=f'IVF-{nlist}-PQ-{m_pq}')

plt.yscale('log')
plt.xlabel('Recall@100 (Точность)')
plt.ylabel('Запросов в секунду (QPS) - Log Scale')
plt.title('Trade-off: Точность vs Скорость')
plt.grid(True, which="both", ls="-", alpha=0.5)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



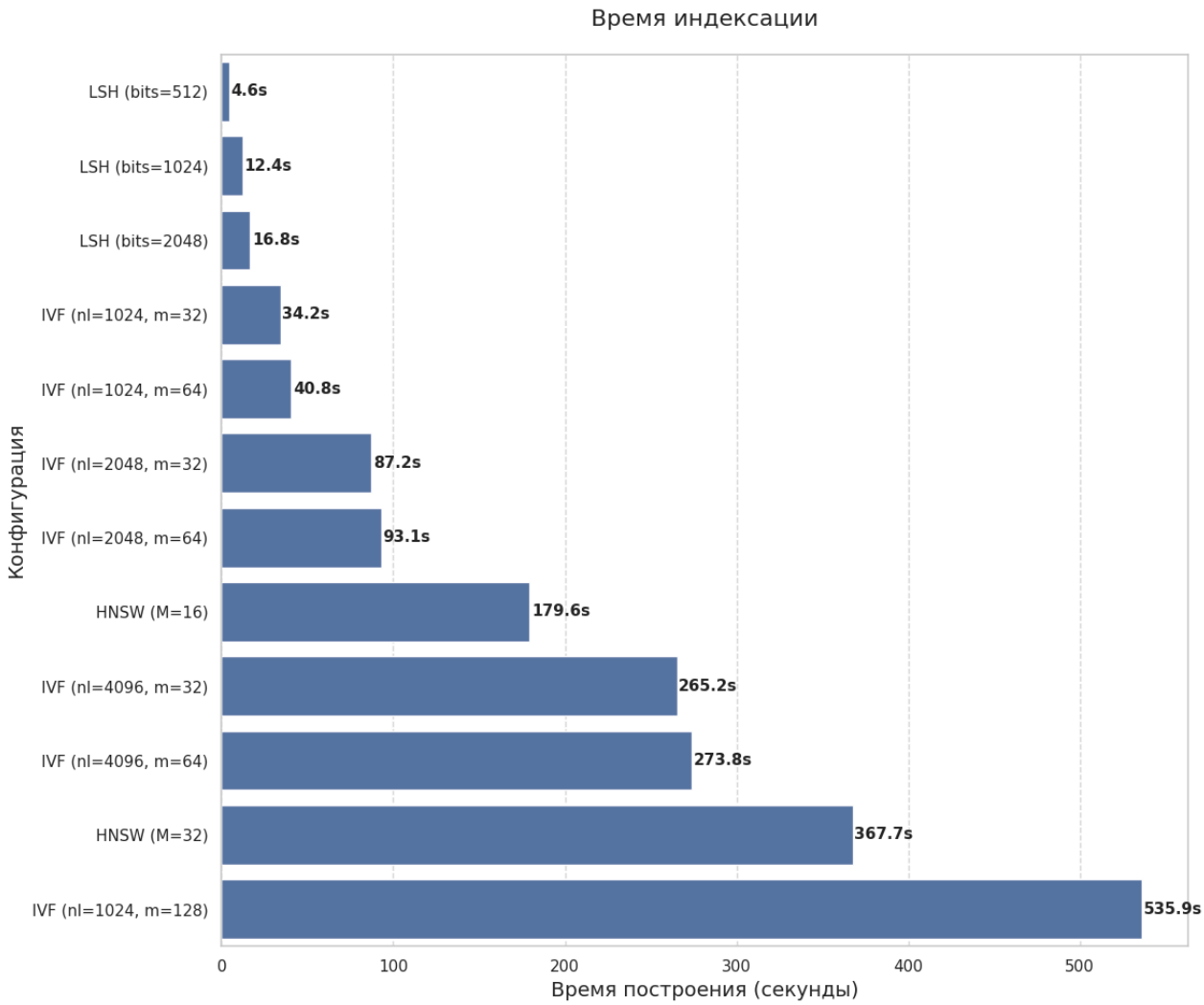
Напишите программный код или сгенерируйте его с помощью искусственного интеллекта.

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

for m in df_hnsw['M'].unique():
    subset = df_hnsw[df_hnsw['M'] == m]
    ax1.plot(subset['efSearch'], subset['recall'], 'o-', label=f'M={m}')
ax1.set_title('HNSW: Влияние efSearch на Recall')
ax1.set_xlabel('efSearch')
ax1.set_ylabel('Recall')
ax1.legend()

for m in df_ivf['m'].unique():
    for nlist in df_ivf['nlist'].unique():
        subset = df_ivf[(df_ivf['m'] == m) & (df_ivf['nlist'] == nlist)]
        ax2.plot(subset['nprobe'], subset['recall'], 's-', label=f'm={m}, nlist={nlist}')

ax2.set_title('IVF+PQ: Влияние nprobe на Recall')
ax2.set_xlabel('nprobe')
ax2.set_ylabel('Recall')
ax2.legend()

plt.show()
```



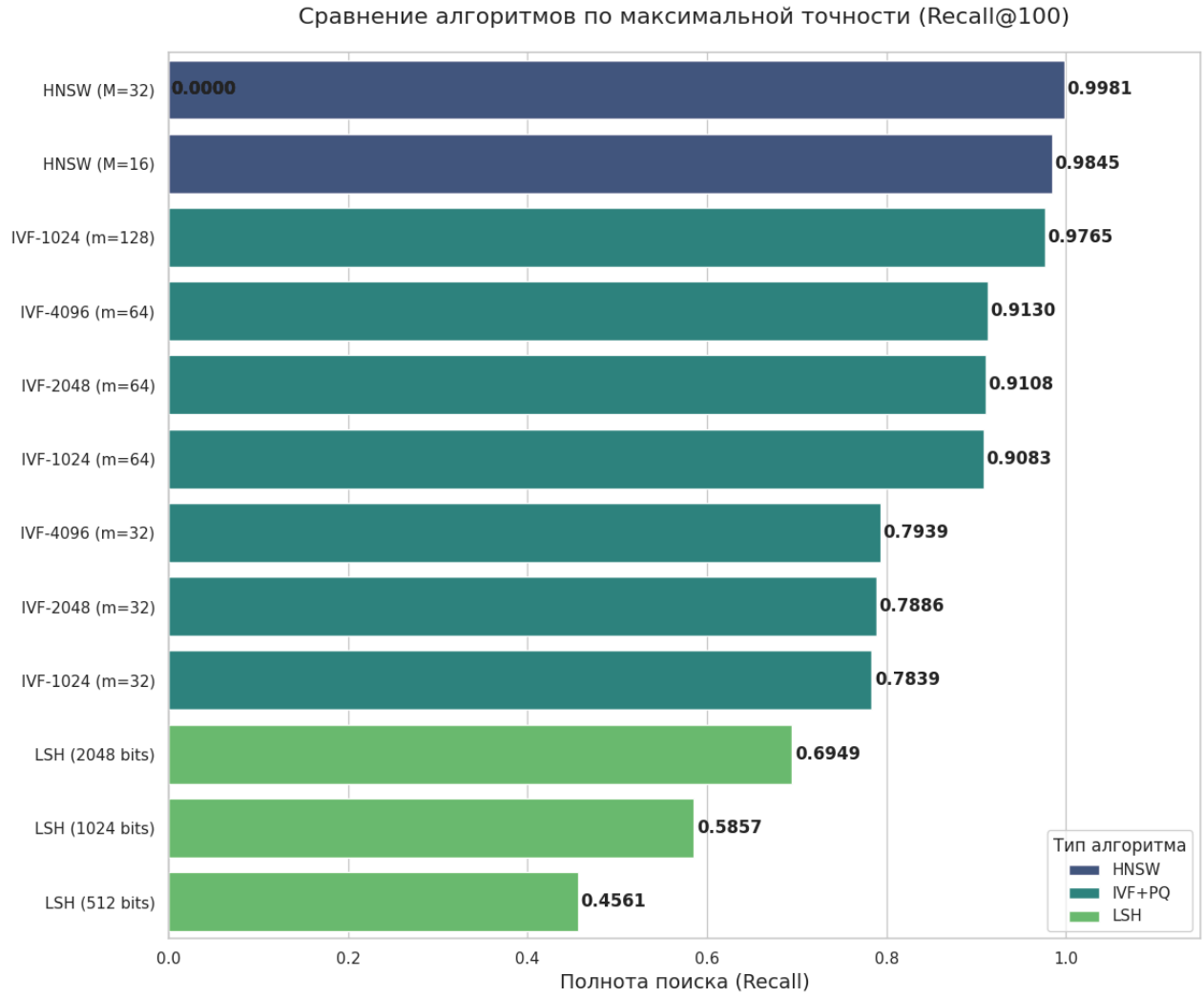



```
lsh_bt = df_lsh[['nbits', 'build_time']].drop_duplicates().copy()
lsh_bt['Config'] = 'LSH (bits=' + lsh_bt['nbits'].astype(str) + ')'

hnsw_bt = df_hnsw[['M', 'build_time']].drop_duplicates().copy()
hnsw_bt['Config'] = 'HNSW (M=' + hnsw_bt['M'].astype(str) + ')'

ivf_bt = df_ivf[['nlist', 'm', 'build_time']].drop_duplicates().copy()
ivf_bt['Config'] = 'IVF (nl=' + ivf_bt['nlist'].astype(str) + ', m=' + ivf_bt['m'].astype(str) + ')'

all_bt = pd.concat([
    lsh_bt[['Config', 'build_time']],
    hnsw_bt[['Config', 'build_time']],
    ivf_bt[['Config', 'build_time']]
]).sort_values('build_time')

plt.figure(figsize=(12, 10))

bar_plot = sns.barplot(x='build_time', y='Config', data=all_bt)

for i, v in enumerate(all_bt['build_time']):
    bar_plot.text(v + 1, i, f"{v:.1f}s", va='center', fontsize=11, fontweight='bold')

plt.title('Время индексации', fontsize=16, pad=20)
plt.xlabel('Время построения (секунды)', fontsize=14)
plt.ylabel('Конфигурация', fontsize=14)
plt.grid(axis='x', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```




```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

lsh_max = df_lsh.groupby('nbits')['recall'].max().reset_index()
lsh_max['Method'] = 'LSH (' + lsh_max['nbits'].astype(str) + ' bits)'
lsh_max['Type'] = 'LSH'
lsh_max = lsh_max.rename(columns={'recall': 'Recall'})

hnsw_max = df_hnsw.groupby('M')['recall'].max().reset_index()
hnsw_max['Method'] = 'HNSW (M=' + hnsw_max['M'].astype(str) + ')'
hnsw_max['Type'] = 'HNSW'
hnsw_max = hnsw_max.rename(columns={'recall': 'Recall'})

ivf_max = df_ivf.groupby(['nlist', 'm'])['recall'].max().reset_index()
ivf_max['Method'] = 'IVF-' + ivf_max['nlist'].astype(str) + ' (m=' + ivf_max['m'].astype(str) + ')'
ivf_max['Type'] = 'IVF+PQ'
ivf_max = ivf_max.rename(columns={'recall': 'Recall'})

df_compare = pd.concat([
    lsh_max[['Method', 'Recall', 'Type']],
    hnsw_max[['Method', 'Recall', 'Type']],
    ivf_max[['Method', 'Recall', 'Type']]
]).sort_values('Recall', ascending=False)


plt.figure(figsize=(12, 10))
sns.set_style("whitegrid")
```

```
plot = sns.barplot(x='Recall', y='Method', data=df_compare, hue='Type', palette='viridis', dodge=False)

for i in plot.patches:
    width = i.get_width()
    if pd.notna(width):
        plot.annotate(format(width, '.4f'),
                      (width, i.get_y() + i.get_height() / 2.),
                      ha='center', va='center',
                      xytext=(25, 0),
                      textcoords='offset points',
                      fontsize=12, fontweight='bold')

plt.xlim(0, 1.15)
plt.title('Сравнение алгоритмов по максимальной точности (Recall@100)', fontsize=16, pad=20)
plt.xlabel('Полнота поиска (Recall)', fontsize=14)
plt.ylabel('', fontsize=14)
plt.legend(title='Тип алгоритма', loc='lower right')

plt.tight_layout()
plt.show()
```




```
def get_index_size_mb(index):
    serialized = faiss.serialize_index(index)
    return len(serialized) / (1024 * 1024)
```

```
import time
import pandas as pd
import faiss
```

```
import faiss

best_configs = [
    {'type': 'HNSW', 'M': 32},
    {'type': 'HNSW', 'M': 16},
    {'type': 'IVF+PQ', 'nlist': 1024, 'm': 128},
    {'type': 'IVF+PQ', 'nlist': 4096, 'm': 64},
    {'type': 'LSH', 'nbits': 2048},
    {'type': 'LSH', 'nbits': 1024}
]

dimension = 128
sizes_results = []

for idx, cfg in enumerate(best_configs, 1):
    t_type = cfg['type']
    start_cfg = time.time()

    if t_type == 'LSH':
        label = f"LSH ({cfg['nbits']} bits)"
        index = faiss.IndexLSH(dimension, cfg['nbits'])
        index.add(base_vectors)

    elif t_type == 'HNSW':
        label = f"HNSW (M={cfg['M']})"
        index = faiss.IndexHNSWFlat(dimension, cfg['M'])
        index.add(base_vectors)

    elif t_type == 'IVF+PQ':
        label = f"IVF-{cfg['nlist']} (m={cfg['m']})"
        quantizer = faiss.IndexFlatL2(dimension)
        index = faiss.IndexIVFPQ(quantizer, dimension, cfg['nlist'], cfg['m'], 8)
        index.train(base_vectors)
        index.add(base_vectors)

    size_mb = get_index_size_mb(index)
    cfg_duration = time.time() - start_cfg

    sizes_results.append({
        'Config': label,
        'Size (MB)': size_mb,
        'Build Time (s)': cfg_duration
    })

    print(f"[{idx}/{len(best_configs)}] {label:22} | Size: {size_mb:7.2f} MB | Build Time: {cfg_duration:6.2f}s")

df_sizes = pd.DataFrame(sizes_results)
```

```
[1/6] HNSW (M=32)             | Size:  747.80 MB | Build Time: 361.28s
[2/6] HNSW (M=16)             | Size:  625.86 MB | Build Time: 179.76s
[3/6] IVF-1024 (m=128)        | Size:  130.33 MB | Build Time: 539.51s
```